

Problem Set 6: The DNA Sequence Marker Finder

Navneet Das

09-09-2026

Header

Course: Coding for AI

Assignment: Problem Set 6

Author: Navneet Das

Source Code

Here is the Python script for the assignment:

```
1 """
2 Author : Navneet Das
3 Date : 09-09-2026
4 Course: Coding for AI
5 Assignment: Problem Set 6
6 """
7
8 def search4letters(phrase: str, letters: str = 'ATCG') -> set:
9     """Return the set of target 'letters' found in a supplied phrase"""
10    return set(letters).intersection(set(phrase))
11
12 def analyze_sequence(sequence_list, marker):
13    print(f" Inside Function (Start) ID: {id(sequence_list)} | Data: {
14    sequence_list}")
15    # In-place modification (mutating the shared object)
16    sequence_list.append(marker)
17    print(f" Inside Function (End) ID: {id(sequence_list)} | Data: {
18    sequence_list}")
19
20 #=====main code
21
22 try:
23     print("\t\t\tThe DNA Sequence Marker Finder")
24     print("-" * 70)
25
26     print("\n--- Task 1: Building search4letters ---")
27     print(f"Positional arguments: {search4letters('TGGACC', 'GC')}")
28     print(f"Keyword arguments: {search4letters(letters='CG', phrase='
29     TGGACC')}")
30     print(f"Defaults: {search4letters('TGGACC')}")
31
32     print("\n--- Task 1: Edge Cases ---")
```

```

30     print(f"Empty phrase: {search4letters('')}")
31     print(f"Mix positional/keyword: {search4letters('TGGACC', letters='
GC')}")
32
33     print(f"\n\n\n|-\{"{'-'*50}\"}-|")
34     print("\n--- Task 2: Tracking Memory References (Core Demo) ---")
35
36     dna_database = ['AATCCG', 'TGGCTA']
37     print(f" Global Scope (Before) ID: {id(dna_database)} | Data: {
dna_database}")
38     analyze_sequence(dna_database, 'CGAT')
39     print(f" Global Scope (After) ID: {id(dna_database)} | Data: {
dna_database}")
40
41     print("\n--- Task 2: The Slice Experiment ---")
42     dna_database_sliced = ['AATCCG', 'TGGCTA']
43     print(f" Global Scope (Before Slice) ID: {id(dna_database_sliced)}
| Data: {dna_database_sliced}")
44     analyze_sequence(dna_database_sliced[:], 'CGAT')
45     print(f" Global Scope (After Slice) ID: {id(dna_database_sliced)} |
Data: {dna_database_sliced}")
46
47     print("\n--- Task 2: Slice Edge Case (Repeated Calls) ---")
48     analyze_sequence(dna_database_sliced[:], 'CGAT')
49
50     print(f"|-\{"{'-'*50}\"}-|")
51
52 except ValueError as ve:
53     print(ve)
54 finally:
55     input("\n\n\n\n\nPress Enter to Exit....")

```

Question to Answer

Question: Why was the `dna_database` modified in the global scope, even though the `analyze_sequence` function never returned anything?

Answer: In Python, variables are like name tags attached to objects in memory. When we pass `dna_database` into the function, we are not making a new copy of the list. Instead, we are just giving the function's parameter (`sequence_list`) a name tag that points to the exact same list in memory.

Because lists are mutable (meaning they can be changed in place), using the `.append()` method alters the list directly where it lives. Since both the global variable and the function parameter are looking at the exact same list, any changes made inside the function are automatically seen outside. We do not need to return anything because we updated the original list directly rather than creating a new one.

Conclusion

This assignment helped me understand how Python handles functions and memory using call by object reference. I learned that when we pass mutable objects like lists into a function, modifying them inside the function modifies them globally. However, if we pass a sliced copy (like `list[:]`), it creates a new separate object, keeping our original

data completely safe. Understanding this difference is very important for managing data correctly in programming.